Docker-03

Task 1.Create a Docker Image from a Running Container

1. Objective

To create a Docker image from an existing running container and use that image to launch a new container.

2. Commands and practical steps

Environment: Ubuntu EC2 instance with Docker installed.

Step 1: Check existing containers

docker ps
docker ps -a

Purpose: `docker ps` displays running containers. `docker ps -a` displays all containers, including stopped ones.

Step 2: Start an Nginx container

docker run -d --name my-nginx -p 8081:80 nginx

Purpose: Downloads the Nginx image if needed and starts a container in the background.

- `-d` — runs in the background.
- `--name` — assigns a container name.
- `-p 8081:80` — maps EC2 port 8081 to container port 80.
- `nginx` — the image used to create the container.

Step 3: Create an image from the running container

docker commit my-nginx my-custom-image:v2

Purpose: Saves the container's current filesystem changes into a new image.

Step 4: Verify the image

docker images

Purpose: Confirms that `my-custom-image:v2` exists locally.

Step 5: Create a second container

docker run -d --name my-nginx-test -p 8082:80 my-custom-image:v2

Purpose: Uses the newly created image to start another container on port 8082.

Step 6: Verify both containers

docker ps

Purpose: Confirms that both containers are running.

Step 7: Test in a browser

http://EC2-PUBLIC-IP:8081
http://EC2-PUBLIC-IP:8082

Purpose: Confirms that Nginx is accessible through both mapped ports. Your EC2 security group must allow the required inbound traffic.

3. How it works in real time

Nginx Docker image

Original template

Running container: `my-nginx`

Port 8081 → container port 80

`docker commit`

Creates a new image from the container's filesystem

New container: `my-nginx-test`

Port 8082 → container port 80

In simple words, an image is a template, and a container is a running instance created from that template.

4. Important points to remember

Command or concept	Meaning
`docker images`	Lists local images
`docker ps`	Lists running containers
`docker ps -a`	Lists all containers

`docker commit`	Creates an image from a container
`docker run`	Creates and starts a container
`-p 8081:80`	Host port 8081 maps to container port 80
`docker logs my-nginx`	Displays container logs
`docker stop my-nginx`	Stops the container

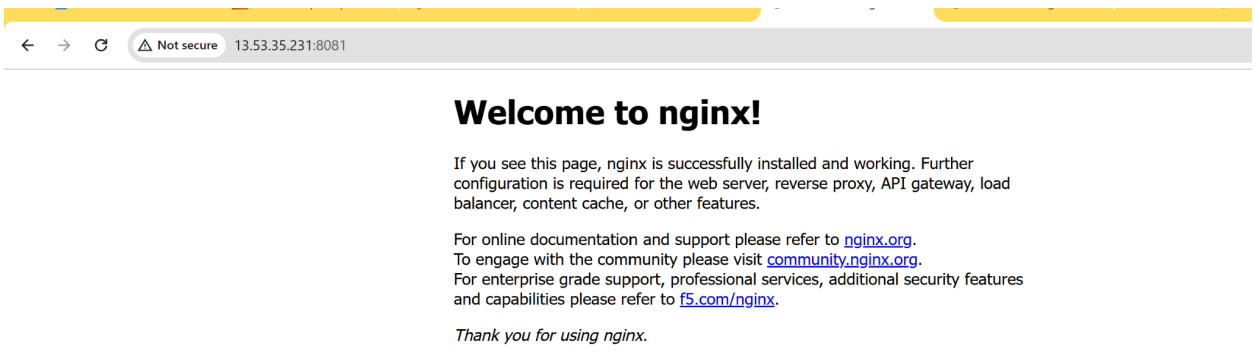
Real-world recommendation: `docker commit` is useful for learning and debugging, but production teams generally build repeatable images using a `Dockerfile` and store them in a container registry such as Amazon ECR or Docker Hub. A commit also does not save data stored in mounted volumes.

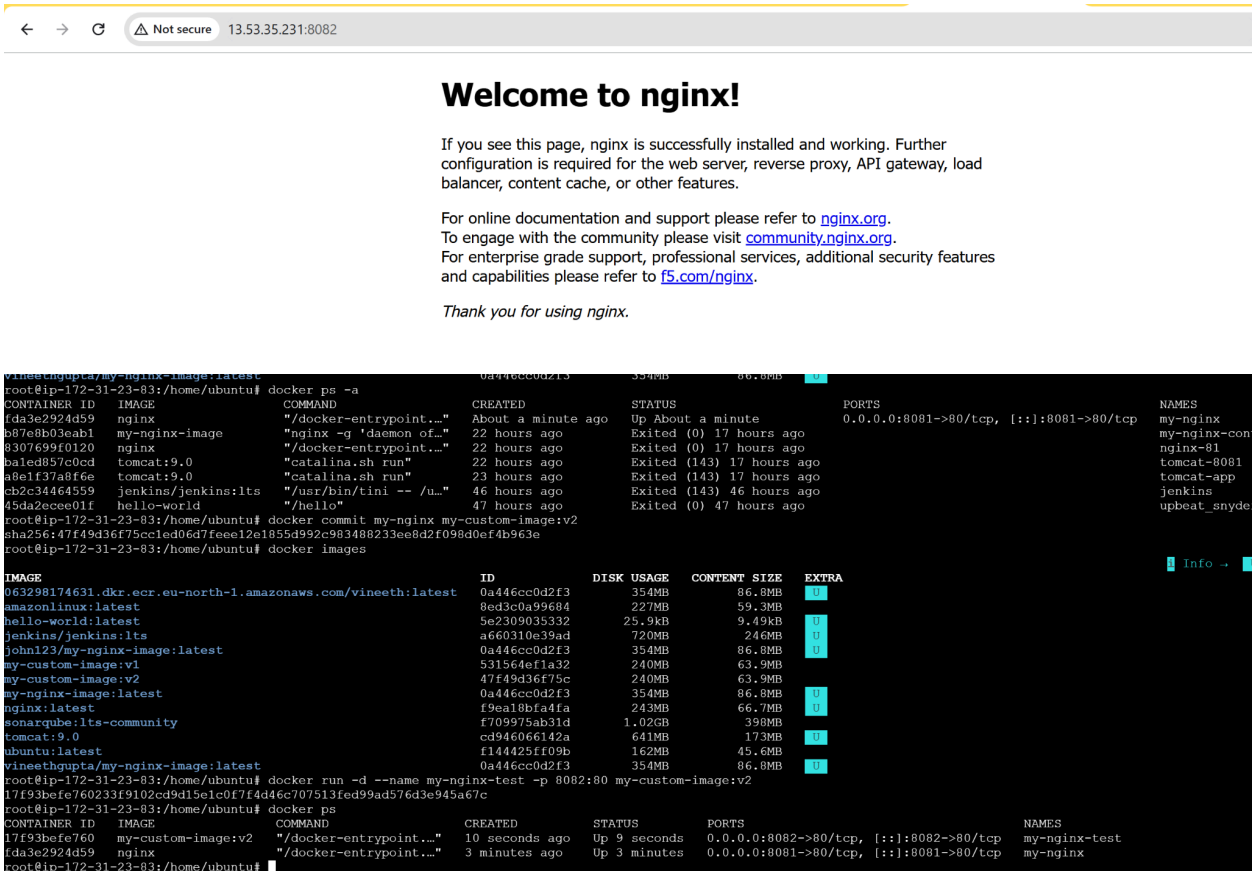
5. Troubleshooting tips

- Container is not running: `docker ps -a`
- Container exits: `docker logs CONTAINER_NAME`
- Port already in use: Choose another host port.
- Browser cannot connect: Check the container status, port mapping, EC2 security group, and network access.
- Image not found: `docker images`

6. Conclusion

Successfully created a custom Docker image from a running Nginx container using `docker commit`, launched a second container from that image, and verified both through browser access.





Task 2: Copy a Docker Image from a Local Machine to a Docker Server and Load It

Objective: To transfer a Docker image from a local Windows machine to a remote Ubuntu EC2 server and load it into Docker.

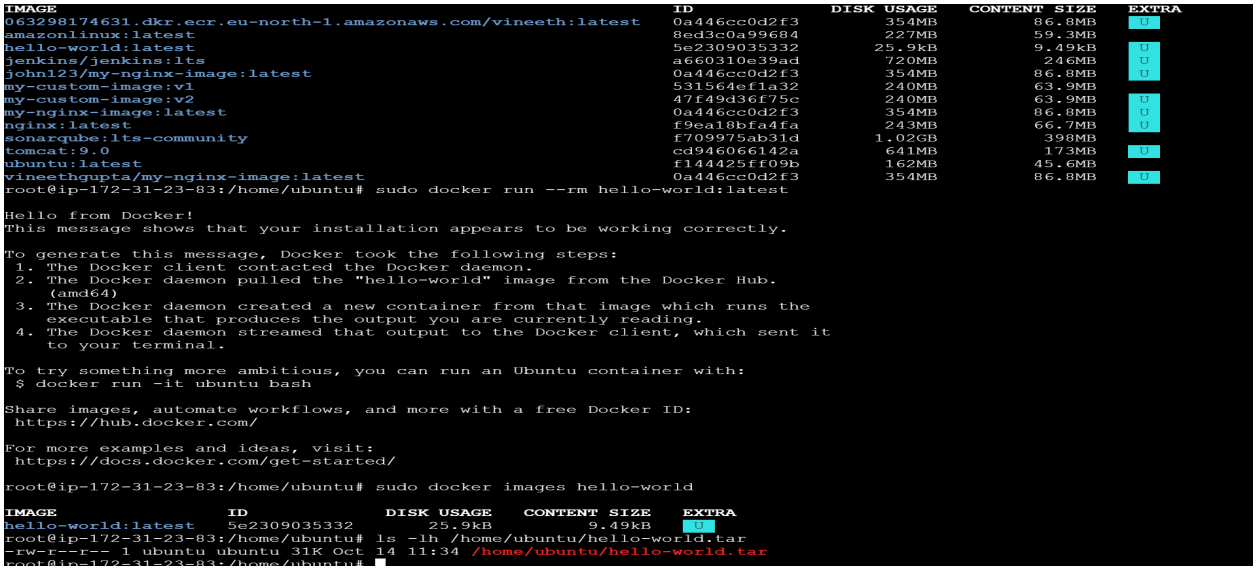
Procedure:

1. Verified the local Docker image using `docker images`.
2. Exported the image to a TAR file using `docker save -o hello-world.tar hello-world:latest`.
3. Transferred the TAR file to EC2 using `scp`.
4. Connected to the Ubuntu EC2 server using SSH.
5. Loaded the image using `sudo docker load -i /home/ubuntu/hello-world.tar`.
6. Verified the image using `sudo docker images`.
7. Executed the image using `sudo docker run --rm hello-world:latest`.

Real-time working: Docker saves the image as an archive, SCP transfers the archive securely to the remote server, and Docker loads the archive into the destination's local image store. A container can then be created from the loaded image.

Result: The image archive was transferred to the EC2 server, and the Hello World container executed successfully.

Conclusion: Successfully practised offline Docker image transfer between a local machine and a remote Docker server.



```
IMAGE                                                                 ID             DISK USAGE   CONTENT SIZE   EXTRA
063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth:latest          0a446cc0d2f3       354MB         86.8MB    U
amazonlinux:latest                                                    8ed3c0a99684       227MB         59.3MB
hello-world:latest                                                    5e2309035332      25.9kB         9.49kB    U
jenkins/jenkins:lts                                                   a660310e39ad       720MB          246MB    U
john123/my-nginx-image:latest                                         0a446cc0d2f3       354MB         86.8MB    U
my-custom-image:v1                                                    531564ef1a32       240MB         63.9MB
my-custom-image:v2                                                    47f49d36f75c       240MB         63.9MB    U
my-nginx-image:latest                                                 0a446cc0d2f3       354MB         86.8MB    U
nginx:latest                                                          f9ea18bfa4fa       243MB         66.7MB    U
sonarqube:lts-community                                               f709975ab31d       1.02GB          398MB
tomcat:9.0                                                            cd946066142a       641MB          173MB    U
ubuntu:latest                                                         f144425ff09b       162MB         45.6MB
vineethgupta/my-nginx-image:latest                                    0a446cc0d2f3       354MB         86.8MB    U
root@ip-172-31-23-83:/home/ubuntu# sudo docker run --rm hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

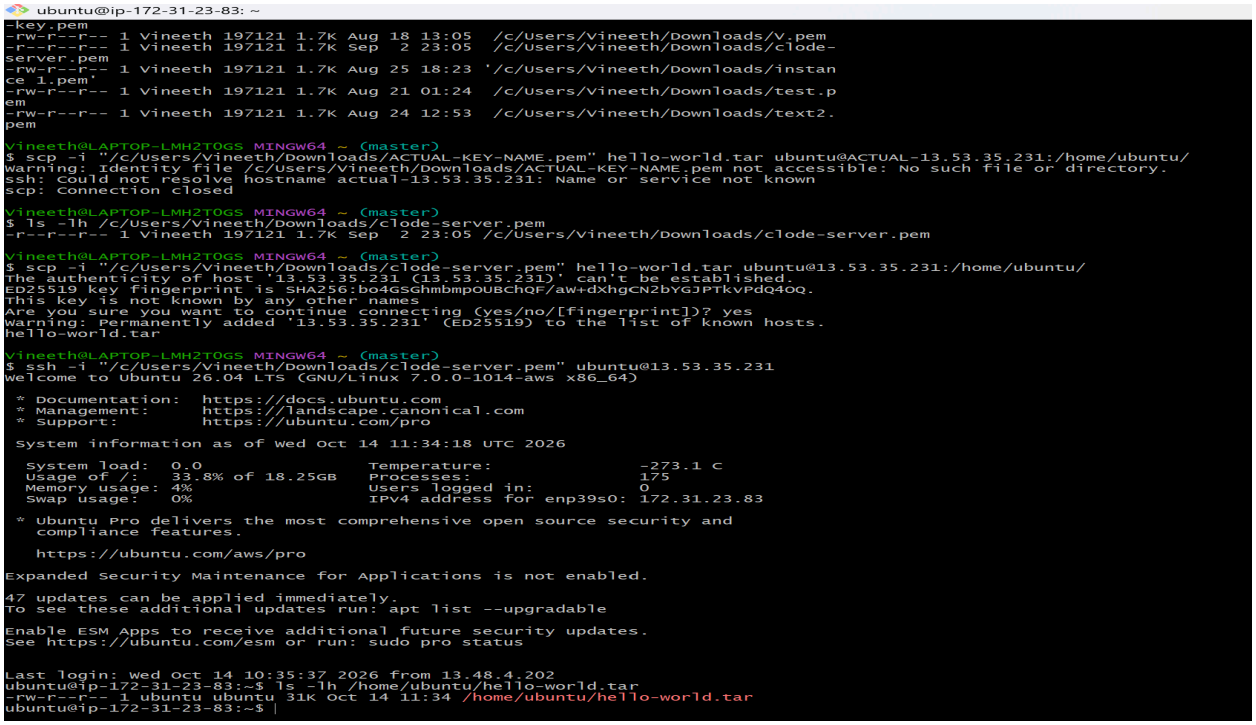
To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

root@ip-172-31-23-83:/home/ubuntu# sudo docker images hello-world

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   5e2309035332       25.9kB         9.49kB    U
root@ip-172-31-23-83:/home/ubuntu# ls -lh /home/ubuntu/hello-world.tar
-rw-r--r-- 1 ubuntu ubuntu 31K Oct 14 11:34 /home/ubuntu/hello-world.tar
root@ip-172-31-23-83:/home/ubuntu#
```



```
ubuntu@ip-172-31-23-83: ~
-key.pem
-rw-r--r-- 1 Vineeth 197121 1.7K Aug 18 13:05  /c/Users/Vineeth/Downloads/V.pem
-r--r--r-- 1 Vineeth 197121 1.7K Sep  2 23:05  /c/Users/Vineeth/Downloads/clode-
server.pem
-rw-r--r-- 1 Vineeth 197121 1.7K Aug 25 18:23 '/c/Users/Vineeth/Downloads/instan
ce 1.pem'
-rw-r--r-- 1 Vineeth 197121 1.7K Aug 21 01:24  /c/Users/Vineeth/Downloads/test.p
em
-rw-r--r-- 1 Vineeth 197121 1.7K Aug 24 12:53  /c/Users/Vineeth/Downloads/text2.
pem

Vineeth@LAPTOP-LMH2TOGS MINGW64 ~ (master)
$ scp -i "/c/Users/Vineeth/Downloads/ACTUAL-KEY-NAME.pem" hello-world.tar ubuntu@ACTUAL-13.53.35.231:/home/ubuntu/
Warning: Identity file /c/Users/Vineeth/Downloads/ACTUAL-KEY-NAME.pem not accessible: No such file or directory.
ssh: Could not resolve hostname actual-13.53.35.231: Name or service not known
scp: Connection closed

Vineeth@LAPTOP-LMH2TOGS MINGW64 ~ (master)
$ ls -lh /c/Users/Vineeth/Downloads/clode-server.pem
-r--r--r-- 1 Vineeth 197121 1.7K Sep  2 23:05 /c/Users/Vineeth/Downloads/clode-server.pem

Vineeth@LAPTOP-LMH2TOGS MINGW64 ~ (master)
$ scp -i "/c/Users/Vineeth/Downloads/clode-server.pem" hello-world.tar ubuntu@13.53.35.231:/home/ubuntu/
The authenticity of host '13.53.35.231 (13.53.35.231)' can't be established.
ED25519 key fingerprint is SHA256:bo4GSGhmbmpOUBChQF/aW+dXhgCN2bYGJPTkvPdQ4oQ.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.53.35.231' (ED25519) to the list of known hosts.
hello-world.tar

Vineeth@LAPTOP-LMH2TOGS MINGW64 ~ (master)
$ ssh -i "/c/Users/Vineeth/Downloads/clode-server.pem" ubuntu@13.53.35.231
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1014-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Oct 14 11:34:18 UTC 2026

  System load:  0.0                Temperature:               -273.1 C
  Usage of /:   33.8% of 18.25GB   Processes:                 175
  Memory usage: 4%                 Users logged in:           0
  Swap usage:   0%                 IPv4 address for enp39s0: 172.31.23.83

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

47 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Wed Oct 14 10:35:37 2026 from 13.48.4.202
ubuntu@ip-172-31-23-83:~$ ls -lh /home/ubuntu/hello-world.tar
-rw-r--r-- 1 ubuntu ubuntu 31K Oct 14 11:34 /home/ubuntu/hello-world.tar
ubuntu@ip-172-31-23-83:~$
```

Create a Docker Image Using Alpine and Customize It with Tomcat

1. Objective

To create a custom Docker image using Alpine Linux as the base image, install Java and Apache Tomcat, and run a Tomcat server inside a container.

Real-time DevOps use: Companies build custom Docker images containing only the required operating system packages and application server. Alpine is a small Linux distribution, so it can help reduce image size.

2. How it works

Alpine Linux

Small base image

Install Java runtime

Tomcat requires Java to run

Install Apache Tomcat

Web/application server

Custom Docker image

Run Tomcat on port 8080

We'll do this on your existing Ubuntu EC2 Docker server, `13.53.35.231`.

3. Practical steps on your EC2 server

You are using Ubuntu EC2 with Docker installed. Run the following commands in your EC2 SSH terminal, not in Windows Git Bash.

Step 1: Create a project directory

```
mkdir -p ~/alpine-tomcat
cd ~/alpine-tomcat
```

Purpose: Keeps the Dockerfile and related files organized in one directory.

Step 2: Create the Dockerfile

Run:

nano Dockerfile

Paste this entire configuration into the editor:

```
FROM alpine:3.22

RUN apk add --no-cache openjdk21-jre curl tar

ENV CATALINA_HOME=/opt/tomcat
ENV PATH="$CATALINA_HOME/bin:{PATH}"

RUN curl -fSL https://downloads.apache.org/tomcat/tomcat-10/v10.1.61/bin/apache-tomcat-10.1.61.tar.gz \
   | tar -xz -C /opt \
   && mv /opt/apache-tomcat-10.1.61 /opt/tomcat

EXPOSE 8080

CMD ["catalina.sh", "run"]
```

Save the file in Nano:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

What this Dockerfile does:

- `FROM alpine:3.22` — uses Alpine Linux as the base.
- `RUN apk add` — installs Java, curl and tar.
- `curl` and `tar` — download and extract Apache Tomcat.
- `ENV` — sets the Tomcat directory and command path.
- `EXPOSE 8080` — documents the port Tomcat uses.
- `CMD` — starts Tomcat when the container runs.

Tomcat 10.1.61 requires Java 11 or later; the Java 21 runtime meets that requirement.



Apache Downloads
+1

Step 3: Build your custom Docker image

sudo docker build -t alpine-tomcat:v1 .

Purpose: Docker reads the `Dockerfile`, downloads Alpine, installs Java, downloads Tomcat and builds your custom image.

The dot (`.`) means the current directory contains the build files.

Wait for the build to finish. The first build may take a few minutes.

Step 4: Verify your image

sudo docker images

Look for:

alpine-tomcat v1

Step 5: Run a container

We'll use host port `8084` to avoid conflicts with your earlier containers.

sudo docker run -d --name alpine-tomcat-app -p 8084:8080 alpine-tomcat:v1

Purpose: Starts Tomcat in the background and maps EC2 port `8084` to Tomcat's port `8080`.

Step 6: Check the container

sudo docker ps

If the container is not running, check its logs:

sudo docker logs alpine-tomcat-app

Step 7: Test Tomcat

First, test from the EC2 server itself:

curl -I http://localhost:8084

A response such as `HTTP/1.1 200` or `HTTP/1.1 302` indicates that the web server is responding.

Then open this URL in your browser:

`http://13.53.35.231:8084`

If it does not open, check the EC2 security group's inbound rules and allow TCP port `8084` from your own IP address.

4. Real-time workflow

Dockerfile

Instructions for building the image

Docker build

Installs Java and Tomcat on Alpine

alpine-tomcat:v1

Custom image

Running container

Port 8084 → port 8080

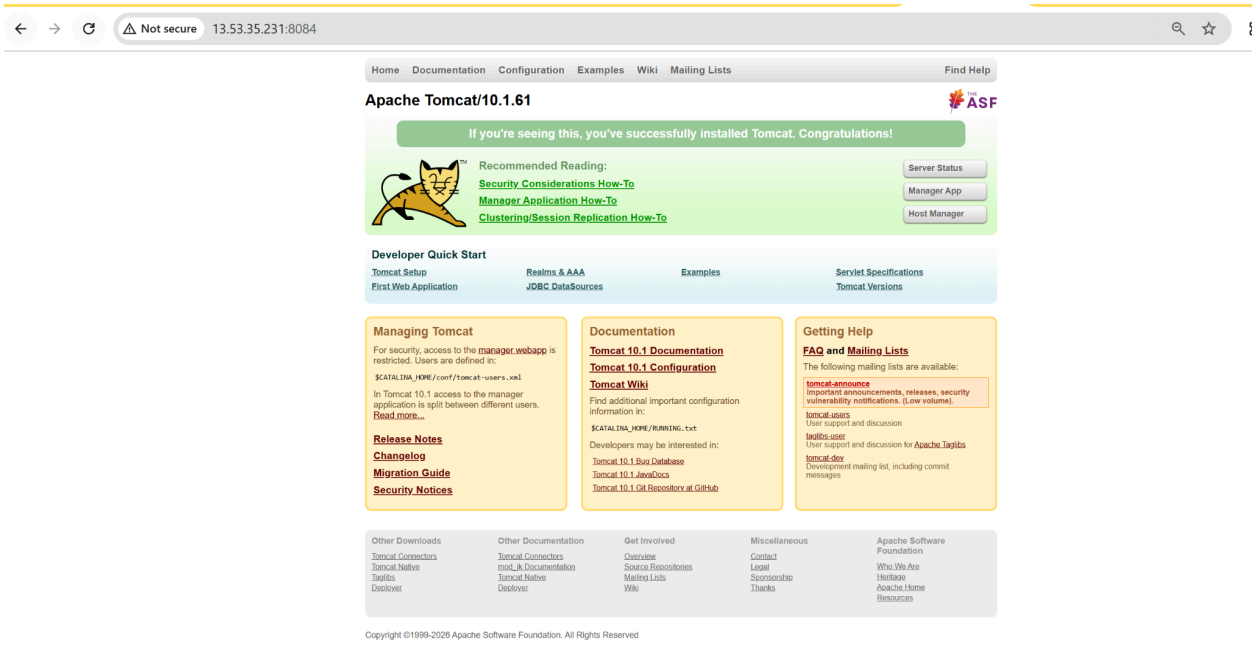
5. Important points for your notes

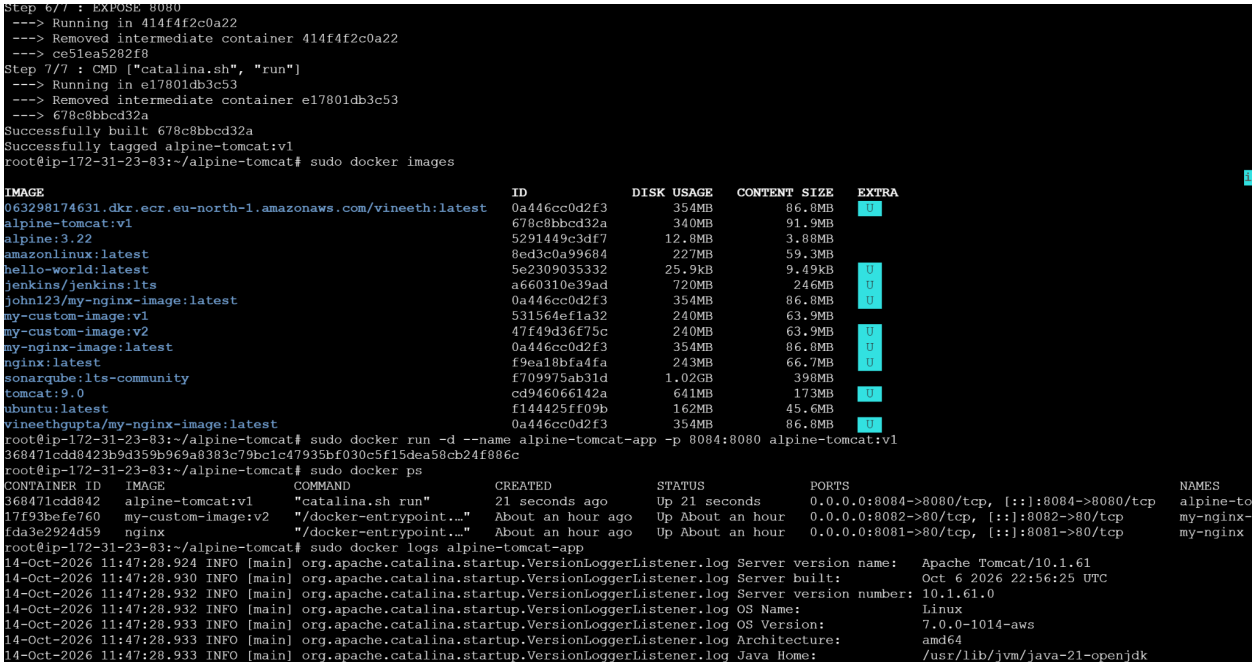
- Alpine is the base operating system.
- Java is required to execute Tomcat.
- Tomcat serves Java web applications.
- Dockerfile contains instructions to create an image.
- Docker image is the packaged template.
- Container is the running instance of that image.
- `EXPOSE 8080` alone does not publish the port; `-p 8084:8080` makes it accessible through the host port.

For production, use a currently supported Alpine base, pin and verify downloaded software, and run the service as a non-root user. This lab uses a root-run container to keep the initial exercise straightforward.

6. Conclusion for your lab record

Created a custom Docker image using Alpine Linux as the base image, installed Java and Apache Tomcat through a Dockerfile, built the image, launched a container, and tested Tomcat through port mapping.





Task 4: Single-Stage and Multi-Stage Dockerfiles

Objective: Build and run a Java application using single-stage and multi-stage Dockerfiles, then compare the image sizes.

Step 1: Connect to EC2 and enter the project

We connected to the Ubuntu EC2 instance and entered the cloned repository.

cd ~/multi-stage-example

Repository: `https://github.com/betawins/multi-stage-example.git`

Step 2: Build the Java application

We installed Maven on EC2 and built the application before creating the Docker images.

mvn clean package -DskipTests

Check the generated JAR:

ls -lh target/multi-stage-example-0.0.1-SNAPSHOT.jar

Purpose: Generate the JAR file required to run the Java application.

Step 3: Create the single-stage Dockerfile

We created `Dockerfile.single` using the Java 8 JRE base image.

```
cat > Dockerfile.single <<'EOF'
FROM eclipse-temurin:8-jre-alpine

WORKDIR /app

COPY target/multi-stage-example-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
EOF
```

Build the single-stage image:

```
sudo docker build -f Dockerfile.single -t java-app-single:v1 .
```

Purpose: Create a Docker image that contains the Java runtime and application JAR.

Step 4: Run the single-stage container

```
sudo docker run -d --name java-single -p 8085:8080 java-app-single:v1
```

The application runs on container port `8080`, mapped to EC2 port `8085`.

Step 5: Create the multi-stage Dockerfile

We created `Dockerfile.multi` with a builder stage and a runtime stage.

```
cat > Dockerfile.multi <<'EOF'
FROM eclipse-temurin:8-jdk-alpine AS builder

WORKDIR /app

COPY target/multi-stage-example-0.0.1-SNAPSHOT.jar app.jar

FROM eclipse-temurin:8-jre-alpine AS runtime

WORKDIR /app

COPY --from=builder /app/app.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
EOF
```

Build the multi-stage image:

sudo docker build -f Dockerfile.multi -t java-app-multi:v1 .

Purpose: Demonstrate separate builder and runtime stages. In our actual Dockerfile, the JAR was already built on EC2, so the builder stage only copied it.

Step 6: Run the multi-stage container

sudo docker run -d --name java-multi -p 8086:8080 java-app-multi:v1

The application runs on container port `8080`, mapped to EC2 port `8086`.

Step 7: Verify both containers

sudo docker ps

We confirmed that both containers were running:

Container	Port mapping	Status
`java-single`	`8085:8080`	Running
`java-multi`	`8086:8080`	Running

Step 8: Check application logs

sudo docker logs java-single
sudo docker logs java-multi

We confirmed that both Spring Boot applications started successfully.

Step 9: Test both applications

curl -I http://localhost:8085/actuator/health
curl -I http://localhost:8086/actuator/health

Result: Both returned `HTTP 200`, confirming that their health endpoints were responding successfully.

The root URL returned `404`, but that did not mean the applications had failed.

Step 10: Compare image sizes

We used:

sudo docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" | grep -E 'REPOSITORY|java-app-single|java-app-multi'

Output:

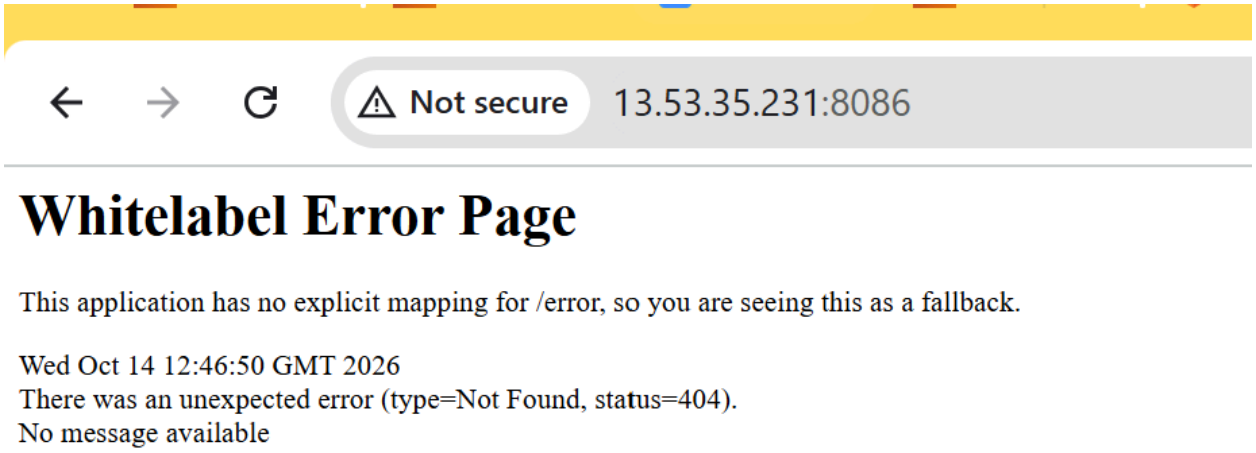
Image	Size
`java-app-single:v1`	**265 MB**
`java-app-multi:v1`	**265 MB**

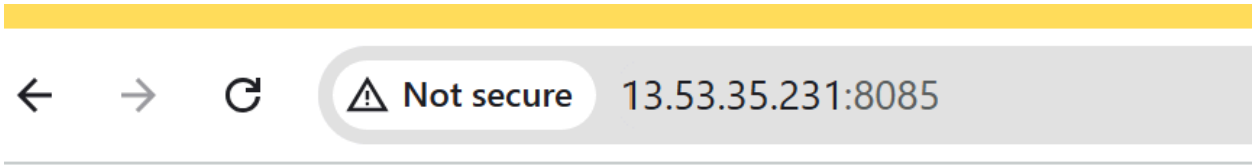
Both images had the same size because they used the same runtime base image and application JAR.

Conclusion

We successfully created two Docker images, ran both containers, checked their logs, tested their health endpoints, and compared their image sizes.

Key learning: Multi-stage builds allow build tools to be kept out of the final runtime image. Our demonstration used a prebuilt JAR, so it did not demonstrate compiling source code inside the builder stage.





Task 5. Install docker compose and execute sample application.

Objective: Install Docker Compose and execute a sample application.

Install Docker Compose
sudo apt update

1. sudo apt install docker-compose-v2 -y
2. Verify installation
   sudo docker compose version
   Result: Docker Compose `2.40.3`.

Create the project directory
mkdir ~/compose-task

3. cd ~/compose-task

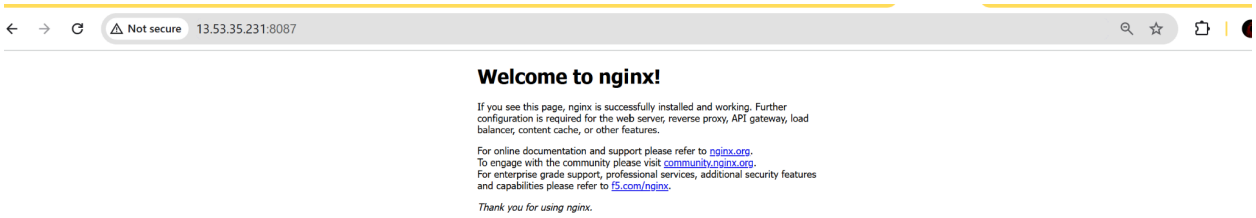
Create `compose.yaml`

```
services:
  web:
    image: nginx:latest
    ports:
      - "8087:80"
```

4. Start the application
   sudo docker compose up -d
5. Check the running container
   sudo docker compose ps
6. Test the application
   curl http://localhost:8087
   Result: `Welcome to nginx!`
7. Access it in the browser
   `http://13.53.35.231:8087`

Conclusion

Docker Compose was installed successfully, and we used a `compose.yaml` file to run an Nginx container in the background. The application responded with the Nginx welcome page.




```
p><em>Thank you for using nginx.</em></p>
/body>
/html>
buntu@ip-172-31-23-83:~/compose-task$ sudo docker compose ps
AME                  IMAGE          COMMAND                  SERVICE   CREATED         ST
ompose-task-web-1    nginx:latest   "/docker-entrypoint.…"   web       3 minutes ago   Up
buntu@ip-172-31-23-83:~/compose-task$ curl http://localhost:8087
!DOCTYPE html>
html>
head>
title>Welcome to nginx!</title>
style>
tml { color-scheme: light dark; }
ody { width: 35em; margin: 0 auto;
ont-family: Tahoma, Verdana, Arial, sans-serif; }
/style>
/head>
body>
h1>Welcome to nginx!</h1>
p>If you see this page, nginx is successfully installed and working.
urther configuration is required for the web server, reverse proxy,
PI gateway, load balancer, content cache, or other features.</p>

p>For online documentation and support please refer to
a href="https://nginx.org/">nginx.org</a>.<br/>
o engage with the community please visit
a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
or enterprise grade support, professional services, additional
ecurity features and capabilities please refer to
a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

p><em>Thank you for using nginx.</em></p>
/body>
/html>
buntu@ip-172-31-23-83:~/compose-task$
```

Task 6: Implement a Solution to Scan Images When Pushed to a Docker Registry

Objective: Install Trivy, push a Docker image to Docker Hub, and scan the image for security vulnerabilities.

Tools required: AWS EC2 (Ubuntu), Docker, Docker Hub, Trivy.

Step 1: Install Docker

If Docker is already installed, skip this step.

sudo apt-get update
sudo apt-get install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker

Verify:

docker --version

Step 2: Install Trivy

Install the required packages:

sudo apt-get update
sudo apt-get install -y wget apt-transport-https gnupg

Add the Trivy repository:

wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo gpg --dearmor -o /usr/share/keyrings/trivy.gpg
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb generic main" | sudo tee /etc/apt/sources.list.d/trivy.list

Install Trivy:

sudo apt-get update
sudo apt-get install -y trivy

Verify:

trivy --version

Step 3: Check available Docker images

docker images

We used the existing `nginx:latest` image on the EC2 server.

Step 4: Log in to Docker Hub

Create an account at Docker Hub if you don't already have one.

Log in from your EC2 terminal:

docker login

Enter your Docker Hub credentials when prompted.

Step 5: Tag the Docker image

We used the existing Nginx image and tagged it with our Docker Hub username.

docker tag nginx:latest vineethgupta/my-nginx:v1

Explanation:

- `nginx:latest` — existing local image.
- `vineethgupta` — Docker Hub username.
- `my-nginx` — repository name.
- `v1` — image tag.

Step 6: Push the image to Docker Hub

docker push vineethgupta/my-nginx:v1

Verify the image in your repository:

Open vineethgupta/my-nginx on Docker Hub

We confirmed that the `v1` tag appeared in the repository.

Step 7: Scan the Docker image using Trivy

Run:

trivy image vineethgupta/my-nginx:v1

Trivy downloads its vulnerability database if needed and scans the image.

The report includes vulnerability details such as:

- CVE ID

- Package name
- Severity
- Vulnerability description
- Reference URL

We successfully ran the scan and received vulnerability results.

Step 8: Scan only HIGH and CRITICAL vulnerabilities

To filter the report to serious vulnerabilities, run:

trivy image --severity HIGH,CRITICAL vineethgupta/my-nginx:v1

To make the command return a failure exit code when such vulnerabilities are found:

trivy image --exit-code 1 --severity HIGH,CRITICAL vineethgupta/my-nginx:v1

This is useful in CI/CD pipelines because it allows a build to fail when serious vulnerabilities are detected.

Final result

Docker installed and images available

Trivy installed and verified

Image pushed to Docker Hub

Image scanned successfully

Conclusion: We installed Trivy on the EC2 server, pushed an Nginx Docker image to Docker Hub, and scanned it for security vulnerabilities. We also learned how to filter HIGH and CRITICAL vulnerabilities.

Practice status: The manual scanning steps are complete. The automatic scanning on every registry push using a Docker Hub webhook and Jenkins pipeline is not yet configured.

Explore / vineethgupta / my-nginx



vineethgupta/my-nginx

By vineethgupta • Updated 4 minutes ago

IMAGE

0 0

Manage Repository

Overview Tags

Sort by Newest Filter tags

TAG

v1

Last pushed **4 minutes** by vineethgupta

```
docker pull vineethgupta/my-nginx:v1
```

Digest	OS/ARCH	Last pull	Compressed size
8792d6624724	linux/amd64	less than 1 day	60.89 MB

hub

V

vineethgupta

Community User

Repositories Starred

Search by repository name

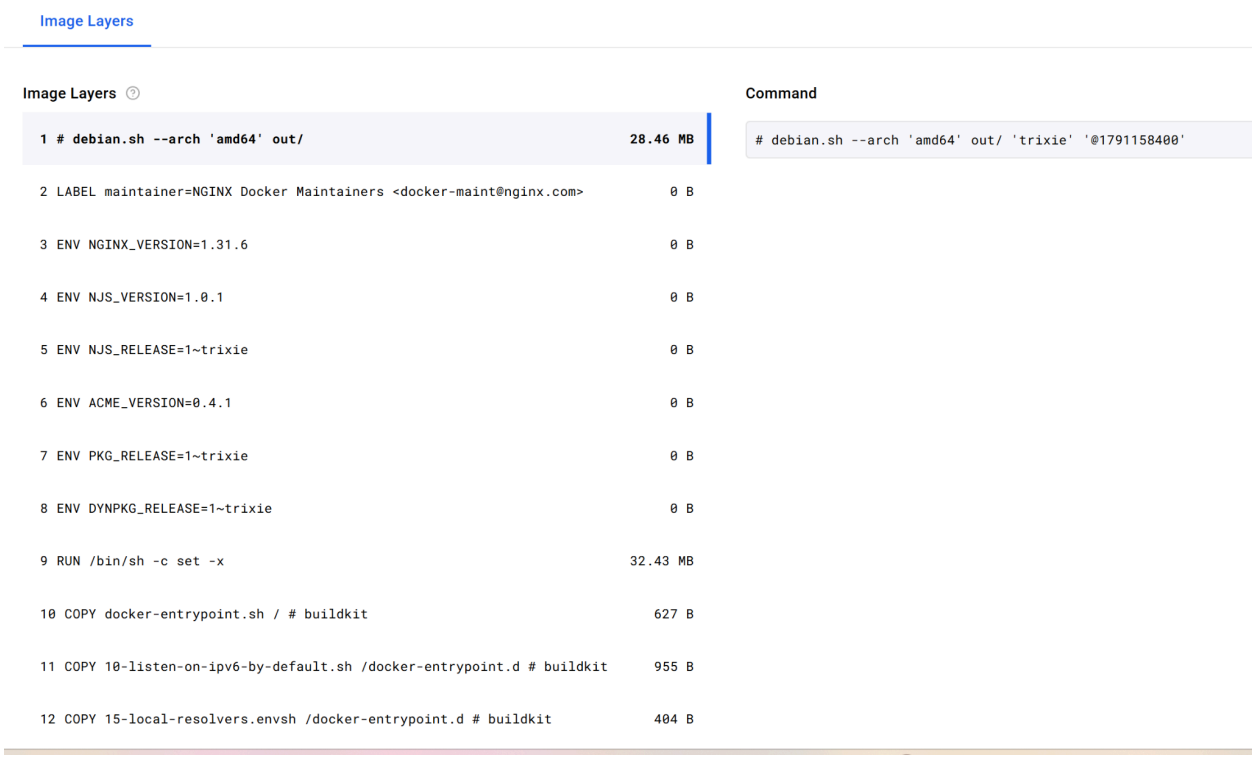
Displaying 1 to 2 of 2 rep

IMAGE

vineethgupta/my-nginx

vineethgupta

12m 12



Task 7: Scan Docker Images When Pushed to AWS ECR

Objective

Automatically scan Docker images for security vulnerabilities when they are pushed to Amazon Elastic Container Registry (ECR).

Steps we completed

Step 1: Open Ubuntu EC2

Connected to your Ubuntu EC2 instance and used the terminal.

Step 2: Verify AWS CLI and Docker

aws --version
docker --version

Step 3: Configure AWS permissions

Created/used an EC2 IAM role with the `AmazonEC2ContainerRegistryPowerUser` policy so the instance could access ECR.

Step 4: Check AWS account

aws sts get-caller-identity

Account ID used: `063298174631`

Region: `eu-north-1`

Step 5: Create or locate the ECR repository

Repository name: `my-nginx`

AWS Console path:

AWS Console → Amazon ECR → Private registry → Repositories → my-nginx

Step 6: Log in to ECR

Run this if you need to log in again:

aws ecr get-login-password --region eu-north-1 | docker login --username AWS --password-stdin 063298174631.dkr.ecr.eu-north-1.amazonaws.com

Step 7: Download the Nginx image

docker pull nginx:latest

Step 8: Tag the image for ECR

docker tag nginx:latest 063298174631.dkr.ecr.eu-north-1.amazonaws.com/my-nginx:v1

Step 9: Push the image to ECR

docker push 063298174631.dkr.ecr.eu-north-1.amazonaws.com/my-nginx:v1

We confirmed that image tag `v1` was uploaded successfully.

Step 10: Enable automatic scanning

In the AWS Console, open:

ECR → Private registry → Repositories → my-nginx → Edit

Find Image scanning settings → Scan on push and turn it on. Your screenshot shows the toggle is blue/on.

Click Save changes if the page still has unsaved changes.

Step 11: Verify the setting from the terminal

```
aws ecr describe-repositories \
  --repository-names my-nginx \
  --region eu-north-1 \
  --query 'repositories[0].imageScanningConfiguration' \
  --output json
```

Expected output:

```
{
    "scanOnPush": true
}
```

Step 12: Check the uploaded image

```
aws ecr describe-images \
  --repository-name my-nginx \
  --region eu-north-1
```

We confirmed the `v1` image existed in ECR.

Step 13: View security scan findings

```
aws ecr describe-image-scan-findings \
  --repository-name my-nginx \
  --image-id imageTag=v1 \
  --region eu-north-1
```

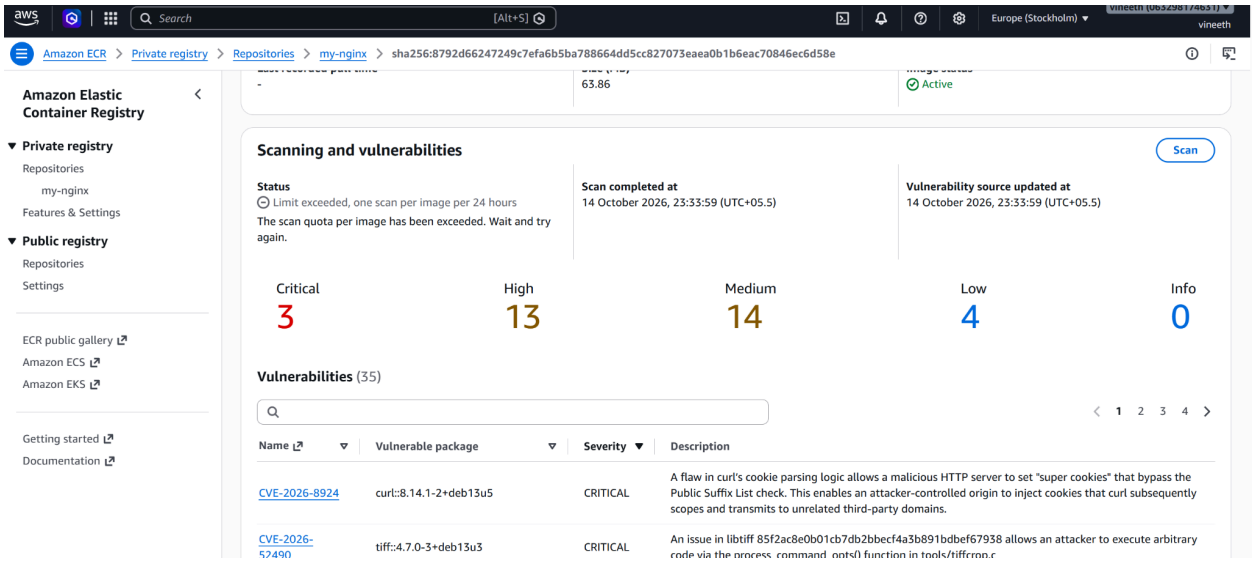
We received vulnerability findings, including a HIGH-severity finding for `libxml2`.

Conclusion for your assignment

Task 7: Implement a solution to scan images when pushed to AWS ECR — completed, provided the Scan on push setting is saved and enabled.

- Docker image pushed to ECR: ✅
- Image exists in the `my-nginx` repository: ✅
- Vulnerability scan findings retrieved: ✅
- Scan on push enabled in the screenshot: ✅

One small note: if your assignment requires proof that scanning happens automatically after a push, push a new image tag after saving the setting and verify its scan status.



Task 8: Jenkins Pipeline to Build a Docker Image and Push to Docker Hub

Objective

To create a Jenkins pipeline that automatically builds a Docker image and pushes it to Docker Hub.

Tools used: AWS EC2, Ubuntu, Docker, Jenkins, and Docker Hub.

Step 1 — Launch an Ubuntu EC2 instance

1. Open the AWS Management Console.
2. Go to EC2 → Launch instance.
3. Select an Ubuntu AMI.
4. Select an instance type suitable for your lab.
5. Configure the security group to allow:
    - SSH — port 22, restricted to your IP.
    - Custom TCP — port 8080, restricted to your IP, for Jenkins.

6. Launch the instance and connect using SSH.

Step 2 — Install Docker on Ubuntu

Run these commands on the Ubuntu EC2 terminal:

```
sudo apt-get update
sudo apt-get install -y docker.io
sudo systemctl enable --now docker
sudo docker --version
sudo docker ps
```

Purpose: Install Docker and verify that the Docker service is running.

Step 3 — Run Jenkins in a Docker container

Run Jenkins using a persistent volume so that jobs and credentials remain saved.

```
sudo docker volume create jenkins_home

sudo docker run -d \
  --name jenkins \
  --restart unless-stopped \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
```

Open Jenkins in your browser:

`http://YOUR_EC2_PUBLIC_IP:8080`

Complete the initial Jenkins setup if required. Install the suggested plugins and create your admin user.

In your lab, Jenkins was already running, so you did not need to perform this installation again.

Step 4 — Configure Docker access for Jenkins

Jenkins runs inside a container, so it needs access to the host's Docker daemon to build images.

Your lab required Docker CLI inside Jenkins and access to the host Docker socket. The working setup uses the existing `jenkins_home` volume and Docker socket mount.

Check that Docker works inside Jenkins:

```
sudo docker exec jenkins docker --version
sudo docker exec jenkins docker ps
```

Expected result: Docker version information and a list of running containers, without a permission error.

Step 5 — Create a Docker Hub account and repository

1. Open Docker Hub.
2. Sign in with your account.
3. Create a repository named `jenkins-demo`.
4. Set the repository visibility as required for your assignment.
5. Create a Docker Hub access token from your account security settings.

For this lab:

- Docker Hub username: `vineethgupta`
- Repository: `vineethgupta/jenkins-demo`

Keep the access token private.

Step 6 — Add Docker Hub credentials to Jenkins

1. Open Jenkins.
2. Go to Manage Jenkins → Credentials.
3. Open System → Global credentials (unrestricted).
4. Click Add Credentials.

Enter the following details:

Field	Value

Kind	Username with password
Username	`vineethgupta`
Password	Your Docker Hub access token
ID	`dockerhub-creds`
Description	Docker Hub credentials

5. Click Create.

Important: The credential ID must exactly match `dockerhub-creds`, because the pipeline uses this ID to log in.

Step 7 — Create a Jenkins Pipeline job

1. From the Jenkins Dashboard, click New Item.
2. Enter the name `DockerHub-Push-Pipeline`.
3. Select Pipeline.
4. Click OK.
5. Scroll down to the Pipeline section.
6. Set Definition to `Pipeline script`.

Step 8 — Enter the pipeline script

Paste the following script into the Script box. Do not include Markdown code fences.

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = 'vineethgupta/jenkins-demo'
```

```
    }

    stages {
        stage('Create Dockerfile') {
            steps {
                sh '''
                    cat > Dockerfile <<'EOF'
FROM nginx:latest
RUN echo '<h1>Hello from Jenkins Pipeline!</h1>' > /usr/share/nginx/html/index.html
EXPOSE 80
EOF
                '''
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t IMAGE_NAME:BUILD_NUMBER .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_TOKEN'
                )]) {
                    sh '''
                        echo "$DOCKER_TOKEN" | docker login \
                          --username "$DOCKER_USER" --password-stdin
                    '''
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh '''
                    docker push IMAGE_NAME:BUILD_NUMBER
                    docker tag IMAGE_NAME:BUILD_NUMBER $IMAGE_NAME:latest
                    docker push $IMAGE_NAME:latest
                '''
```

```
            }
        }
    }

    post {
        always {
            sh 'docker logout || true'
        }
        success {
            echo 'Docker image pushed successfully!'
        }
    }
}
```

Click Save.

Step 9 — Run the pipeline

1. Open `DockerHub-Push-Pipeline`.
2. Click Build Now.
3. Click the build number under Build History.
4. Click Console Output.
5. Check that all stages complete successfully.

The pipeline performs these operations:

1. Create Dockerfile

Creates the instructions for the image.

2. Build Docker Image

Builds an Nginx image containing a custom welcome page.

3. Login to Docker Hub

Uses Jenkins credentials to authenticate securely.

4. Push to Docker Hub

Uploads both the build-number tag and the latest tag.

Step 10 — Verify the Docker image

Open your repository:

https://hub.docker.com/r/vineethgupta/jenkins-demo

Under Tags, verify that you can see:

- A build-number tag, such as `5`
- `latest`

You can also verify the image from the Ubuntu terminal:

sudo docker pull vineethgupta/jenkins-demo:latest

Troubleshooting

Error	Solution
`docker: not found`	Install Docker CLI inside the Jenkins container or use a custom Jenkins image containing the CLI.
`permission denied` on Docker socket	Verify the socket is mounted and the Jenkins process has permission to access it.
`Could not find credentials entry`	Create the credential in Global credentials with ID `dockerhub-creds`.
`denied: requested access to the resource is denied`	Verify the Docker Hub username, access token, and repository name.
Push stage skipped	Check the first error in Console Output; a previous stage failed.

Security note: Access to `/var/run/docker.sock` effectively grants powerful control over the host's Docker daemon. Use this setup for your lab and avoid exposing the Docker socket to untrusted jobs.

Result

Task completed successfully. The Jenkins pipeline created a Dockerfile, built the Docker image, authenticated to Docker Hub using Jenkins credentials, and pushed the image with both a build-number tag and a `latest` tag.

Short explanation for class:

“I created a Jenkins pipeline to automate Docker image creation and upload. Jenkins generated a Dockerfile, built an Nginx image, logged in to Docker Hub using stored credentials, and pushed the image to the repository. This demonstrates basic CI/CD automation using Jenkins and Docker.”

